k6 and performance testing interview questions, answered with this project

These are performance-testing and k6 questions that come up in SDET, QA lead and performance engineer interviews. Each answer explains the idea and shows an example. Most answers end with an **In the project** line that points to the file in this repository that does it, or says why the project does not use it. Links go to github.com/niranjankagri/k6-performance-testing.

Versions: k6 2.3.0, Node.js 24.

Contents

1. Performance testing basics (Q1-Q12)
2. Workload modelling (Q13-Q20)
3. k6 fundamentals (Q21-Q32)
4. Scenarios and executors (Q33-Q39)
5. Checks, thresholds and metrics (Q40-Q50)
6. Test data, correlation and realism (Q51-Q56)
7. Results, analysis and bottlenecks (Q57-Q64)
8. CI/CD and environments (Q65-Q70)
9. Scenario questions (Q71-Q78)
10. Quick reference

1. Performance testing basics

Q1. What is performance testing, and why is it more than "load testing"?

Performance testing measures how fast, how stable and how scalable a system is under a given workload. Load testing is one type of it. Others ask different questions: where is the limit (stress), what happens in a burst (spike), and what happens over hours (soak). The goal is a decision: can we release, how many servers do we need, what do we fix first.

Q2. Name the main test types and the question each one answers.

Type	Load shape	Question
Smoke	minimal (1-2 users), short	do the system and the script work at all?
Load	expected peak, held	do we meet the SLOs at normal peak?
Stress	increasing in steps, past the peak	where is the limit, and how do we fail?
Spike	sudden jump, then drop	do we survive a burst and recover?
Soak / endurance	normal, for hours	do memory, queues or response times degrade over time?
Breakpoint	keep increasing until it breaks	what is the absolute capacity?
Scalability	repeat at 1, 2, 4 instances	does capacity grow with resources?

In the project: one file per type in k6/tests.

Q3. What are SLIs, SLOs and SLAs?

- **SLI** (indicator): something you measure, for example the p95 response time of checkout or the error rate.
- **SLO** (objective): the target for an SLI, for example "p95 of checkout under 400 ms".
- **SLA** (agreement): a contract with a customer, with consequences if it is missed. SLOs are usually stricter than the SLA, to keep a safety margin.

In the project: the SLOs are k6 thresholds in thresholds.js.

Q4. Why percentiles instead of the average?

Response times are skewed: most requests are fast and a few are very slow. The average hides that slow tail, and it is the tail that users complain about. p95 means 95% of requests were at least this fast; p99 shows the worst 1%. The maximum is often a single outlier, so it is useful to look at, but a poor thing to set a target on.

Q5. Latency, response time, throughput: what is the difference?

- **Response time** is what the client waits: from sending the request to receiving the full response. In k6 this is `http_req_duration`, which is sending + waiting + receiving.
- **Latency** is often used for the waiting part only (server time + network). k6's `http_req_waiting` is the time to first byte (TTFB).
- **Throughput** is the work done per unit of time: requests per second (`http_reqs` rate) or business transactions per second (orders/s).

Q6. What is Little's Law, and how do you use it?

`concurrency = throughput × response time` (L = λ × W). Some uses:

- **Sizing a test.** To start 2 orders per second when one order journey lasts 8 s (including think time), about 2 × 8 = 16 VUs are busy at once.
- **Sizing** `preAllocatedVUs` in arrival-rate executors.
- **Spotting saturation.** If throughput stays flat while concurrency grows, response time must be growing: requests are queueing.

In the project: the `buyers` scenario in load.js uses this to size `preAllocatedVUs`. The capacity estimate in the README is 8 connections / 0.09 s ≈ 90 orders/s.

Q7. What is a "knee" in a response-time chart?

The point where response time stops being flat and climbs sharply as load increases. Below the knee, the system has spare capacity. At the knee, some resource is saturated (CPU, connections, threads, locks) and requests begin to queue. The knee is the practical capacity limit, usually well below the point where errors start.

In the project: the stress test found it at about 80-120 orders/s; see the "before" screenshot in the README.

Q8. What is the difference between a closed and an open workload model?

- **Closed:** a fixed number of users. Each sends the next request only after the previous answer arrived. When the system slows down, the load drops on its own (coordinated omission).
- **Open:** requests arrive at a rate that does not depend on the system's speed, like independent internet users. When the system slows down, more requests are in flight, which is what happens in production.

Measuring "orders per second" needs an open model. See Q34.

Q9. What is coordinated omission?

In a closed model, the slow responses delay the next requests, so the test sends fewer requests exactly when the system is slow. The measurements then under-represent the slow period, and the results look better than reality. Open-model executors (arrival rate) avoid this, because they keep starting iterations on schedule.

Q10. What should a performance test plan contain?

- Goals, and the questions to answer.
- The SLOs.
- The workload model: user types, mix, rates and think time, taken from production analytics or business forecasts.
- Test types and durations.
- Test data.
- The environment, and how it differs from production.
- Monitoring: which server metrics to watch.
- Entry and exit criteria.
- Risks.
- How results are reported.

Q11. Why must the test environment be close to production?

Capacity depends on hardware, configuration, data volume (an index that works on 1,000 rows may not on 10 million), caches, network and third-party services. Results from a smaller environment can be scaled only with care. It is often better to compare *relative* changes (this release vs the last) than absolute numbers.

In the project: the app runs on the CI runner itself. The README states that these numbers catch regressions but are not production capacity.

Q12. Where does performance testing belong in the delivery process?

Everywhere, at different sizes:

- a smoke-sized performance check on every pull request;
- a load test nightly or before a release;
- stress, soak and capacity tests before major launches or architecture changes;
- production monitoring afterwards, which closes the loop.

This is "shift left" for performance.

2. Workload modelling

Q13. How do you find out what load to simulate?

Use production data where you can:

- access logs or an APM (requests per endpoint, peak hour, peak minute);
- analytics (user journeys, conversion rates);
- business forecasts (a campaign, Black Friday, growth).

Without production data, use the business's expectations and then add headroom.

Q14. What is a transaction mix, and how do you model it in k6?

The share of each user type or journey, for example 90% browsing and 10% buying. In k6, give each journey its own **scenario** with its own executor and rate, so the mix is explicit and each part can be read on its own.

In the project: the `browsers` and `buyers` scenarios in load.js.

Q15. What is think time, and what happens without it?

The pause a real user makes between actions: reading, typing. Without think time, every VU fires requests continuously. 20 VUs then make as many requests as hundreds of real users, and the results answer a different question than intended.

Use random think time (for example 1-3 s), so that VUs do not move in lockstep.

In the project: `think()` in flows.js, scaled with `THINK`. The stress test runs with `THINK=0`, because it is about throughput.

Q16. What is pacing?

Controlling how often an iteration *starts*, rather than pausing inside it. In k6 the arrival-rate executors do this: `rate: 2, timeUnit: '1s'` starts 2 iterations per second, however long each takes.

Q17. Ramp-up: why not start at full load?

- A real system warms up: caches fill, connection pools grow, the JIT compiles.
- Starting at full load creates an artificial spike, which is a different test.
- A ramp shows how response times change as load grows. If something fails, you see at which level it failed.

Q18. How long should a load test run?

Long enough to reach a steady state and collect enough samples for stable percentiles. That is typically 10-60 minutes at the target load, plus ramps. A soak test runs for hours (4-24) to expose leaks. CI versions are shorter rehearsals.

In the project: `PROFILE=ci` (2 minutes) vs `PROFILE=full` (20 minutes; a 4-hour soak) through `byProfile()` in config.js.

Q19. Should every VU use the same test user?

No. One user means one cart, one session and contention that real traffic does not have, or cached answers that real traffic would not get. Use a pool of users, one per VU or per iteration.

In the project: `userForVu(__VU)` in data.js.

Q20. How many VUs do you need for a target of N requests per second?

Use Little's Law: VUs ≈ rate × iteration duration. For an open model, set `preAllocatedVUs` to that estimate and `maxVUs` higher, to absorb slowdowns. Then check `dropped_iterations` after the run: if it is not zero, k6 ran out of VUs and the target rate was not reached.

3. k6 fundamentals

Q21. What is k6, and how does it work?

k6 is an open-source load-testing tool from Grafana Labs, written in Go. You write the tests in JavaScript (ES modules), and k6 runs them in its own embedded JavaScript runtime (Sobek). That runtime is **not Node.js and not a browser**: there is no `require`, no npm packages that need Node APIs, and no DOM. Each VU is a separate runtime running the script's default function (or a scenario's `exec` function) in a loop. The HTTP calls are made by Go code, so one machine can run thousands of VUs.

Q22. What are the stages of a k6 test's life cycle?

1. **init**: top-level code, once per VU (and once more to read `options`). Load files here with `open()` and `SharedArray`. HTTP calls are not allowed here.
2. **setup()**: once, before the load starts. It can make HTTP calls, and whatever it returns is passed to the VU functions and to `teardown()`.
3. **VU code**: `default` or the scenario's `exec` function, repeated by every VU.
4. **teardown()**: once, at the end.
5. **handleSummary()**: once, with all the metrics, to write the summary outputs.

In the project: `setup()` checks the app's health, and `teardown()` in soak.js logs the memory growth.

Q23. Why is a VU's module-level variable "per VU"?

Every VU has its own JavaScript runtime. A `let token` at module level therefore exists once per VU, and it keeps its value across that VU's iterations. This makes it a natural place for a session token: log in once per VU, then reuse the token. VUs share nothing except `SharedArray` data.

In the project: `ensureLoggedIn()` in flows.js.

Q24. What is a SharedArray, and why use it?

Normally each VU gets its own copy of any data loaded in the init stage: 1,000 VUs means 1,000 copies of a CSV file. A `SharedArray` is built once, from a function, and shared read-only by all VUs, which saves a lot of memory. It can only be created in the init stage.

In the project: the users from `data/users.csv` in data.js.

Q25. How do you pass configuration into a k6 script?

Use environment variables, read as `__ENV.NAME`. They can be set with `k6 run -e NAME=value` or in the shell. CLI flags (`--vus`, `--duration`) and `K6_*` environment variables override `options` for the simple cases.

In the project: `BASE_URL`, `PROFILE` and `THINK` in config.js.

Q26. What does `group()` do?

`group()` names a block of steps. Checks and metrics get a `group` tag, so the summary lists checks per group, and k6 records a `group_duration` metric per group. It is good for structuring journeys. Note that a group's duration includes think time.

In the project: `browse` and `buy` groups in flows.js.

Q27. What is the `name` tag, and why do URLs with ids need it?

k6 tags every request with its URL. `/api/products/7`, `/api/products/8` and so on would each become a separate time series, which is a problem known as high cardinality: it is useless for analysis and heavy on memory. Setting `tags: { name: 'GET /api/products/:id' }` folds them into one series. That series can then have its own threshold.

In the project: every function in api.js.

Q28. How do you correlate dynamic values (tokens, ids)?

Read them from one response and use them in the next request: `res.json('token')`, `order.json('id')`. For HTML responses, use `res.html().find(...)` or a regular expression. Correlation is the most common reason why a recorded script fails on replay.

In the project: the login token and the order id in `buy()`.

Q29. Can k6 use npm packages?

Only packages that need no Node APIs, and they must be bundled into a single file (for example with esbuild or webpack). Many helpers are available as remote modules from jslib.k6.io (for example k6-summary, k6-utils), which are imported by URL. k6 extensions (xk6) add Go-written modules, for example for Kafka or SQL; recent k6 versions can resolve them automatically from `k6/x/...` imports.

In the project: `textSummary` from `https://jslib.k6.io/k6-summary/0.1.0/index.js` in summary.js.

Q30. What does `fail()` do, and when do you use it?

`fail(message)` throws an error that ends the current iteration. The error is counted, and the VU continues with the next iteration. Use it when the rest of a journey makes no sense, for example after a failed login. In `setup()`, it stops the whole test.

In the project: `ensureLoggedIn()` and `requireHealthyApp()`.

Q31. Can k6 test browsers?

Yes. The `k6/browser` module drives Chromium and measures Web Vitals (LCP, CLS, INP). Browser VUs cost far more resources than protocol-level VUs. The usual design is therefore many protocol VUs for the load, plus a few browser VUs that measure what the user experiences under that load.

In the project: not used, because this project is about API capacity (see framework-design.md).

Q32. k6, JMeter, Gatling, Locust: how do you choose?

Tool	Scripts	Strengths
k6	JavaScript in git	small CLI, thresholds as code, CI-friendly, Grafana integration, browser module
JMeter	XML test plans (GUI)	protocols galore (JDBC, JMS, FTP...), huge plugin ecosystem, familiar to many teams
Gatling	Scala/Java/Kotlin DSL	very efficient, good HTML reports, strong for JVM teams
Locust	Python	simple, distributed, good for Python teams

Choose the one the team can maintain and review, and that supports the protocols you need.

4. Scenarios and executors

Q33. What is a scenario?

A named workload inside one test, with its own executor, VUs or rate, duration, `exec` function, tags and start time. Several scenarios run in parallel, which is how a test mixes user types (Q14) or adds a monitoring loop (Q50).

Q34. Which executors are there, and when do you use each?

Executor	Model	Use for
`shared-iterations`	fixed total iterations shared by VUs	"run this N times", data processing
`per-vu-iterations`	each VU runs N iterations	each user does exactly N journeys
`constant-vus`	closed	steady fixed concurrency (soak visitors)
`ramping-vus`	closed	stages of concurrency (load, spike)
`constant-arrival-rate`	open	a fixed rate of new iterations (orders per second)
`ramping-arrival-rate`	open	rates in stages (stress, breakpoint)

In the project: `ramping-vus` and `constant-arrival-rate` in load.js and spike.js; `constant-vus` in soak.js; `ramping-arrival-rate` in stress.js.

Q35. What are `preAllocatedVUs` and `maxVUs`?

In arrival-rate executors, k6 needs free VUs to start each new iteration. `preAllocatedVUs` are created before the test starts. If they are all busy because iterations got slower, k6 creates more, up to `maxVUs`. Creating VUs during the test costs time, so the pre-allocated number should cover the normal case (Little's Law).

Q36. What are dropped iterations?

In an arrival-rate executor, an iteration that should have started but could not, because no free VU was left (`maxVUs` reached). The `dropped_iterations` metric counts them. A non-zero value means the test did not deliver the load it claimed. Watch it, or put a threshold on it.

In the project: `dropped_iterations: ['count==0']` in load.js and soak.js.

Q37. What is `gracefulStop` / `gracefulRampDown`?

The time a running iteration is given to finish when its scenario ends, or when VUs are ramped down. Iterations still running after that are interrupted. This is why the `buy` journey clears the cart at its start: an interrupted iteration may leave items behind.

Q38. How do you run one scenario after another?

Use `startTime` on a scenario, for example `startTime: '5m'`, to start it after the previous one has finished. You can also put separate questions in separate test files and run them in sequence in CI.

Q39. How do you give different scenarios different thresholds?

Tag-filtered thresholds. k6 tags every metric with `scenario`, so `'http_req_duration{scenario:buyers}': ['p(95)<300']` applies only to the buyers. The same works for any tag (`name`, `group`, custom tags).

5. Checks, thresholds and metrics

Q40. Check or threshold: what is the difference?

- A **check** is an assertion on one response: status 200, a field present, the total correct. A failed check does **not** fail the test. It is counted in the `checks` rate metric.
- A **threshold** is a pass/fail criterion on a metric over the whole test, for example `p(95)<400` or `rate<0.01`. A missed threshold makes k6 exit with code 99.

Put a threshold on `checks` (for example `rate>0.99`) to make failed checks fail the run.

Q41. Show a threshold with options.

```
thresholds: {
  http_req_failed: ['rate<0.01'],
  'http_req_duration{name:GET /api/products}': ['p(95)<200'],
  checkout_duration: ['p(95)<400', 'p(99)<600'],
  dropped_iterations: ['count==0'],
  checks: [{ threshold: 'rate>0.95', abortOnFail: true, delayAbortEval: '20s' }],
}
```

`abortOnFail` stops the test as soon as the threshold is crossed. `delayAbortEval` waits before evaluating, so that the first few seconds cannot abort the run.

In the project: thresholds.js and the stress thresholds in stress.js.

Q42. What exit code does k6 return when a threshold fails, and why does that matter?

Exit code **99**. CI treats any non-zero exit code as a failed step, so thresholds turn a load test into a gate, without any extra script parsing the results.

In the project: the regression demo exits 99 because checkout p95 is 425 ms against an SLO of 400 ms.

Q43. What built-in metrics does k6 collect?

- **HTTP:** `http_reqs`, `http_req_duration` (and its parts `blocked`, `connecting`, `tls_handshaking`, `sending`, `waiting`, `receiving`), `http_req_failed`.
- **Execution:** `iterations`, `iteration_duration`, `vus`, `vus_max`, `dropped_iterations`.
- **Other:** `checks`, `data_sent`, `data_received`, `group_duration`.

Q44. What are the four custom metric types?

Type	Records	Example
Counter	a sum	`orders_placed`
Gauge	the latest value (plus min/max)	queue length
Rate	the percentage of non-zero values	`order_success`
Trend	statistics: avg, min, max, percentiles	`checkout_duration`

`new Trend('name', true)` marks the values as times, so they are shown in ms.

In the project: all of them except Gauge, in flows.js and soak.js.

Q45. What counts as a failed request in `http_req_failed`?

By default, any status outside 200-399, and network errors. If some 4xx responses are *expected* (for example a 404 test), declare them with `http.setResponseCallback(http.expectedStatuses(200, 404))` or per request with `responseCallback`. Otherwise they inflate the error rate.

Q46. Why are business metrics useful next to HTTP metrics?

HTTP metrics say "requests were fast". Business metrics say "orders worked". A checkout can return 201 quickly with a wrong total, or return a fast 503. Counters and rates in business terms (orders placed, orders refused, success rate) make results understandable to the product owner, and catch outcomes that status codes miss.

In the project: `order_success`, `orders_refused` and `order_handled`. The check `'order: total matches the cart'` verifies the data, not just the status.

Q47. How do you check a response body efficiently under load?

Parse once and check a few key fields. `res.json('path')` reads one value. Avoid deep validation of large bodies in every iteration, because it costs the load generator CPU and lowers the load it can produce. Full schema validation belongs in functional tests. Use `discardResponseBodies` when the body is not needed at all.

Q48. How do you write the end-of-test output yourself?

Export `handleSummary(data)`. It receives every metric and threshold result, and it returns a map of `{ 'path or stdout': content }`.

In the project: summary.js writes:

- the console summary (`textSummary`);
- the full JSON;
- a Markdown table, which CI appends to the run summary page.

Q49. What reports and outputs can k6 produce?

- the console summary and `handleSummary` files;
- the **web dashboard**, live in the browser or exported as one HTML file with `K6_WEB_DASHBOARD_EXPORT`;
- real-time outputs with `--out`: JSON or CSV files, InfluxDB, Prometheus remote write, OpenTelemetry, Grafana Cloud k6, and others.

These real-time outputs feed Grafana dashboards next to the server metrics.

In the project: the HTML dashboard per test (CI artifacts), plus the Markdown and JSON.

Q50. How do you correlate the load with server-side resources?

- Watch the server's metrics (CPU, memory, GC, connections, queue lengths, DB) during the test, in an APM or in Grafana, on the same time axis as the k6 results.
- In small setups, a scenario can poll a health or metrics endpoint and record the values as custom metrics.

In the project: the `server_monitor` scenario in soak.js records `/health`'s `rssMb` every 10 s, with a memory budget threshold.

6. Test data, correlation and realism

Q51. How do you handle test data for a load test?

- **Pre-seeded data**: enough users and products for every VU (CSV or JSON in a `SharedArray`).
- **Unique values where they must be unique**, for example with `__VU` and `__ITER` or random suffixes.
- **Data that is consumed** (for example one-time vouchers) needs one item per iteration, or the test must create it in `setup()`.
- **Reset or clean up** between runs, so that run 10 does not test a database 10 times bigger than run 1 (unless that is the point).

Q52. What is cache warming, and should a test include it?

The first requests fill caches (application, database, CDN), which makes them slower than the steady state. Either warm up before measuring (a ramp, or exclude the first minutes from the analysis), or measure the cold start on purpose. Know which one you report.

Q53. Why log in once per VU and not once per iteration?

A real session lasts many page views. Logging in on every iteration would put far more load on the authentication service than production does, and it would distort the other endpoints' numbers. Test login capacity separately, if that is the question.

In the project: `ensureLoggedIn()` keeps one token per VU.

Q54. How do you avoid VUs moving in lockstep?

Random think time, random data and ramps instead of jumps. Otherwise all VUs send the same request at the same moment, which causes artificial spikes.

Q55. Should static resources (images, CSS) be part of an API load test?

Usually not. A CDN serves them, and loading them from the load generator mostly measures your own network. Test the origin's dynamic endpoints. A browser test, or a separate CDN check, covers the static parts.

Q56. How do you load test an API that needs an OAuth token?

Get the token in `setup()` (one shared service token) or once per VU (user tokens), and refresh it before it expires. Do not include the identity provider in the load unless it is part of the system under test, and agree that with its owners first.

7. Results, analysis and bottlenecks

Q57. How do you read a stress test result?

1. Find the **knee**: the load level at which p95 starts to climb, and compare it with the throughput. If throughput is flat while VUs or response time rise, something is saturated.
2. Find **where errors start** and of which kind: timeouts, 5xx, connection refused.
3. Check **recovery**: after the load drops, do response times return to the baseline?
4. Match all of this with the server metrics to name the bottleneck.

In the project: "Finding" in the README, with charts from before and after the fix.

Q58. What are typical bottlenecks?

- CPU.
- Memory and GC pauses.
- Database: slow queries, missing indexes, locks, a connection pool that is too small.
- Thread or worker pools.
- External calls without timeouts.
- Synchronous logging.
- Network bandwidth.
- Rate limits.
- The load generator itself (Q60).

Q59. What is load shedding, and why is it better than an unbounded queue?

Refusing work quickly (`503` with `Retry-After`) when the system is already full. With an unbounded queue, every request waits longer and longer, timeouts pile up, clients retry, and in the end *everyone* gets a bad experience. With shedding, most users are served at normal speed and some are told to retry: the system degrades gracefully.

In the project: the stress test found unbounded queueing at checkout. `MAX_WAITING` in server.js now refuses new checkouts when 40 are already waiting. The stress test asserts "every order placed or refused with 503" and a bounded p95.

Q60. How do you know the load generator is not the bottleneck?

Watch its CPU and memory. Above about 80% CPU, the measured times include the generator's own delays. Signs of a saturated generator:

- `http_req_blocked` rises;
- iteration rates do not match the plan;
- `dropped_iterations` rises without the server being busy.

The fixes are to distribute the load (several machines, k6 Cloud, the k6 operator on Kubernetes), discard response bodies you do not need, and simplify the checks.

Q61. What does a slowly rising response time in a soak test suggest?

Something that grows with time or with the number of requests:

- a memory leak, which also brings longer GC pauses;
- a growing cache or in-memory collection;
- table growth without an index;
- an accumulating queue;
- log files filling a disk;
- connections that are never released.

Compare the memory trend with the number of processed requests.

In the project: QuickShop keeps every order in memory. The soak test watches `server_rss_mb` against a budget.

Q62. Results differ between two identical runs. What do you do?

- Run each test several times and look at the spread, not at a single number.
- Make sure the environment is stable: no other load, the same data volume, the same configuration, warm caches.
- Compare percentiles, not maxima.
- Use longer steady phases.
- Treat a difference as real only when it is bigger than the run-to-run noise.

Q63. How do you compare a new release with the previous one?

Run the same test, with the same data and environment, for both versions (a baseline). Then compare per-endpoint percentiles and error rates, either with the stored JSON summaries or in Grafana. Set thresholds from the baseline plus a tolerance, for example "p95 no more than 10% worse".

Q64. What goes into a performance test report?

- The goal and the questions.
- The environment and the version under test.
- The workload model.
- The results per SLO (pass or fail).
- Charts: response time and throughput over time, the knee.
- The bottleneck and the evidence for it.
- Risks.
- Recommendations: fix this, add capacity there, retest.

Put the conclusion first, then the evidence.

8. CI/CD and environments

Q65. How do you put performance tests into a pipeline?

- Lint and functional tests first.
- A smoke-sized performance test as the gate on every pull request.
- Load, stress and spike tests nightly or before a release.
- Soak tests on a schedule.
- Thresholds decide pass or fail (exit code 99).
- Results go to the run summary, and the reports are kept as artifacts.

In the project: perf.yml.

Q66. Are results from CI runners trustworthy?

Shared runners are noisy: other jobs, CPU throttling, no control over the hardware. They are fine for catching **regressions** with generous thresholds, and for small loads. Capacity numbers need dedicated load generators and a production-like environment.

In the project: this is stated in the README. The `ci` profile is small, and the `full` profile is meant for a dedicated environment.

Q67. How do you start the system under test inside a CI job?

Start it in the background and wait until it is healthy before the test begins (a health endpoint with retries). Or use a service container (Docker), or deploy to a test environment first. Never start the load before the app is ready, or the first minute measures the start-up.

In the project: `node app/server.js &`, then `curl --retry-connrefused .../health`, and `requireHealthyApp()` in `setup()`.

Q68. How do you install k6 in GitHub Actions?

Use `grafana/setup-k6-action` with a pinned `k6-version`, then call `k6 run`. Alternatives are `grafana/run-k6-action`, the k6 Docker image, or a package manager.

Q69. How do you keep test runs from interfering with each other?

- Give each run its own environment or its own copy of the app.
- Do not run parallel load tests against the same shared target.
- Isolate the data (separate users or tenants).
- In CI, give each test type its own job and runner.

In the project: load, stress and spike each start their own QuickShop on their own runner.

Q70. When is it ethical and safe to run a load test?

Only against systems you own, or have written permission to test, and with the system's owners informed. Never against third-party services in your journeys, such as payment providers or identity providers: use their sandboxes or mocks. Load tests against public sites look like a denial-of-service attack.

In the project: this is why the app under test lives in the repository.

9. Scenario questions

Q71. The load test passes but users complain about slowness. What do you check?

- Does the workload model match reality: the mix, the peak, think time, data volume?
- Were the slow endpoints part of the test at all?
- Was the environment comparable to production?
- Are the users slow because of the frontend (rendering, large bundles), which an API test does not see?
- Are third parties slow in production?
- Do the percentiles in production monitoring match the test's?

Q72. p95 doubled after a release, but the averages look the same. What does that tell you?

The slow tail got worse while typical requests stayed the same. Likely causes:

- a code path that only some requests take (certain data, a cache miss, a lock);
- GC pauses;
- a new external call with an occasional timeout.

Compare per-endpoint percentiles to find which request type changed.

Q73. How would you find the maximum orders per second the system can handle?

Use a stress or breakpoint test with `ramping-arrival-rate`, in steps. Hold each step long enough to settle. Watch throughput (orders completed per second), p95 and errors per step, together with the server resources. Capacity is the highest step that still meets the SLOs: usually below the point where errors begin.

In the project: stress.js steps through 40, 80 and 120 orders/s. The SLOs held up to about 80 orders/s.

Q74. Your test shows errors only at high load: 502s from the load balancer. Where do you look?

The upstream servers: timeouts, crashes, worker or thread exhaustion, health checks failing under load and the load balancer removing instances. Then the load balancer's own connection limits. Compare the time of the first 502s with the server metrics.

Q75. The business plans a campaign with 10x traffic. What do you do?

1. Agree the expected peak and the journey mix with the business.
2. Run a load test at 10x in a production-like environment.
3. Find the bottleneck and fix it or scale it.
4. Run a spike test, because campaigns start suddenly.
5. Plan the extras:
    - autoscaling rules and warm-up;
    - rate limits and load shedding;
    - a feature switch to turn off expensive features;
    - monitoring and alerts for the day.

Q76. A soak test fails after 3 hours with out-of-memory errors. How do you investigate?

1. Confirm the trend: memory rises steadily with requests, not just once.
2. Take heap dumps at intervals and compare them, to see which objects grow.
3. Check the caches without limits, the in-memory collections, listeners that are never removed, and the connection pools.
4. Reproduce it faster with a higher rate.
5. Fix the cause, then rerun the soak test.

Q77. How do you test a microservice that depends on five others?

Decide on the question first:

- **The capacity of this service alone:** mock or stub the dependencies with realistic latencies (for example WireMock or a stub service).
- **The end-to-end system:** test it all together in a full environment.

Contract tests keep the mocks honest. Never send load to services owned by another team without their agreement.

Q78. How would you explain a performance result to a product owner?

Start in business terms: "at the expected Black Friday peak, 99% of customers can pay within 0.4 s. Above twice that peak, some customers are asked to retry; nobody gets an error page." Then name the risk and the cost of fixing it. Keep the charts as evidence, not as the message.

10. Quick reference

```
k6 run script.js                                run (exit 99 if a threshold fails)
k6 run -e PROFILE=full -e BASE_URL=... script.js   pass environment variables
k6 run --vus 10 --duration 30s script.js        override simple options
k6 run --summary-mode=full script.js            more detail in the end-of-test summary
K6_WEB_DASHBOARD=true K6_WEB_DASHBOARD_EXPORT=report.html k6 run script.js   HTML report
k6 run --out json=results.json script.js        every data point, for later analysis
k6 inspect script.js                            show the resolved options
```

Concept	Where in the project
Scenarios, executors, open vs closed	`k6/tests/load.js`, `stress.js`, `spike.js`, `soak.js`
Thresholds (SLOs), abortOnFail	`k6/lib/thresholds.js`, `k6/tests/stress.js`
Checks, groups, custom metrics, think time, per-VU login	`k6/lib/flows.js`
Name tags	`k6/lib/api.js`
SharedArray test data	`k6/lib/data.js`
Profiles (ci / full)	`k6/lib/config.js`
handleSummary: console, JSON, Markdown	`k6/lib/summary.js`
Server monitoring during a test	`soak.js` (`server_monitor` scenario)
Load shedding (the stress-test finding)	`app/server.js`
CI gate	`.github/workflows/perf.yml`